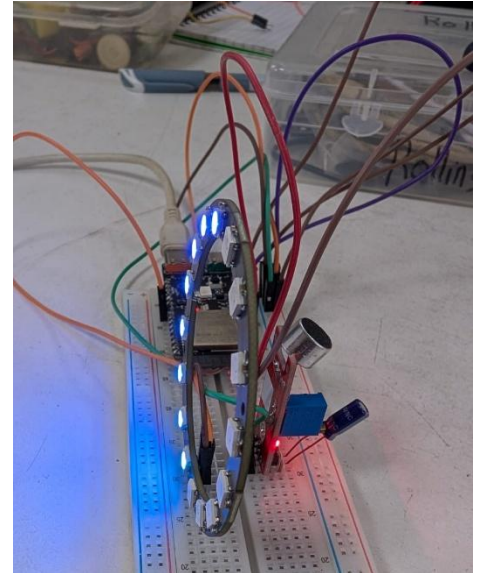


dB Meter + Audio Visualizer Playbook

I. Challenge

Develop a mixed-signal Digital Signal Processing (DSP) pipeline that captures real-time acoustic waveforms, transforms time-domain data into frequency spectra, and computes relative sound pressure levels in decibels (dB). The system must cleanly separate sensitive analog sensor signals from high-current digital switching components while mapping volume parameters to a responsive, dynamic RGB visualizer ring.



II. Guide

A. Concept

To process real-time audio and drive an addressable LED grid smoothly, we rely on these core embedded and DSP principles:

- **1. High-Speed Fixed-Interval Acquisition:** To accurately capture audio waves, an Analog-to-Digital Converter (ADC) must sample the incoming signal at a uniform frequency. To resolve frequencies up to a certain limit, the sampling frequency must double that target (known as the Nyquist boundary).
 - Learn more: [Understanding the Nyquist-Shannon Sampling Theorem](#)
- **2. Dynamic DC Offset Cancellation:** Electret microphones ride on a constant baseline DC voltage reference. To isolate true sound waves, the software calculates the arithmetic mean across a batch of samples and subtracts it from each reading, centering the waveform neatly at zero.
 - Learn more: [DC Bias and Signal Conditioning in AC Audio Circuits](#)
- **3. Root-Mean-Square (RMS) & Logarithmic Scale Mapping:** Isolated background noise spikes can skew simple peak measurements. Computing the Root-Mean-Square (V_{RMS}) yields the true continuous electrical power of the wave, which is then scaled logarithmically to match human auditory perception:

$$\text{dB} = 20 \times \log_{10}(V_{\text{RMS}} + 0.1) + \text{Offset}_{\text{calibration}}$$

- Learn more: [Measuring Sound and Audio Signals in Decibels](#)
- **4. Fast Fourier Transform (FFT) Resolution:** To view the audio by its individual pitch bands, time-domain data is run through a radix-2 FFT algorithm after applying a Hamming window to suppress boundary edge leaks. The width of each frequency bin depends on the sample count (N) and the sampling frequency (f_s):

$$\Delta f = \frac{f_s}{N}$$

- Learn more: [The Fourier Transform Explained](#)
- **5. HSV Color Space Transitions:** Standard RGB manipulation makes creating smooth color flows difficult. By driving the visualizer ring using Hue-Saturation-Value (HSV) dynamics instead, the code can fluidly slide the color phase space around the spectrum purely based on audio intensity.
 - Learn more: [Working with NeoPixel RGB LEDs](#)

Real-World Example: Professional studio VU meters, graphic equalizers on consumer audio receivers, and smart voice assistants (like Amazon Echo or Google Nest). These systems process microphone arrays, apply noise-canceling software filters, and light up reactive LED rings to show voice tracking or volume metrics.

B. Components

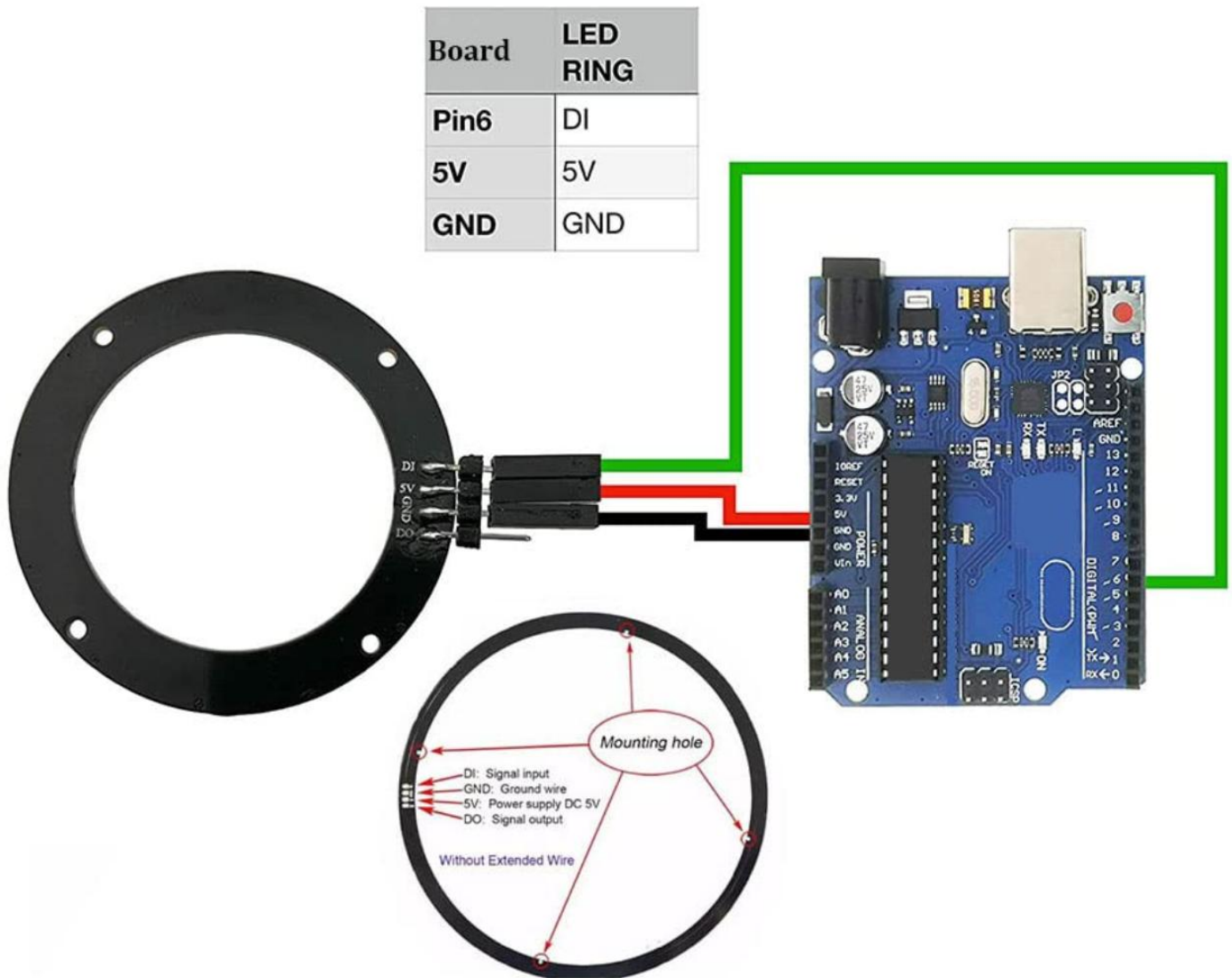
- **Microcontroller:** WeAct Studio ESP32-C6 (32-bit RISC-V architecture).
- **Acoustic Transducer:** LM393 Electret Microphone Module.
- **Visual Interface:** 16-element WS2812B addressable NeoPixel RGB LED Ring.
- **Hardware Filtering:** A 10 μ F to 100 μ F electrolytic capacitor (used to eliminate breadboard power rail noise).

C. Connections

Because addressable LEDs pull rapid current surges that can corrupt sensitive analog readings, the power and data configurations must be strictly managed:

Source Module Pin	Target MCU Pin (ESP32-C6)	Operational Description	Circuit Requirements / Notes
LM393 Mic VCC	V5 / VIN (+5V)	Main Sensor Power Input	Connect to +5V to enforce optimal electret bias and signal swing.
LM393 Mic GND	GND	Ground Reference	Shared ground plane.
LM393 Mic OUT	GPIO 1	Time-Domain Analog Data	Keep this signal path as short as possible to avoid noise pickup.
NeoPixel 5V	V5 / VIN (+5V)	High-Current Power Rail	Place a decoupling capacitor (10 μ F–100 μ F) across Mic VCC and GND.
NeoPixel GND	GND	Ground Reference	Shared ground plane.
NeoPixel DI	GPIO 18	High-Speed Control Stream	Delivers the 800 kHz digital data stream to update the pixels.

How to connect your RGB LED Ring Module explained by this reference diagram:



D. Code

This implementation features data acquisition timing locks, automatic DC offset removal, logarithmic decibel scaling, and an interactive HSV lighting renderer:

```
1. C++
2. /**
3.  * @file Audio_Processor_Visualizer.ino
4.  * @brief Integrated Audio DSP Pipeline (RMS dB, FFT, and HSV Rainbow Visualization)
5.  * @hardware WeAct Studio ESP32-C6, LM393 Microphone Module, 16-LED WS2812B Ring
6.  */
7.
8. #include <Adafruit_NeoPixel.h>
9. #include "arduinoFFT.h"
10.
11. // Hardware Pin Configuration
12. #define MIC_PIN      1
13. #define LED_PIN      18
14. #define NUM_LEDS     16
15.
16. // DSP Sampling Parameters
17. #define SAMPLES      128           // Must be a power of 2 (Radix-2 FFT requirement)
18. #define SAMPLING_FREQ 10000       // 10kHz Sample Frequency (Nyquist boundary = 5kHz)
19. double vReal[SAMPLES];
```

```

20. double vImag[SAMPLES];
21. unsigned long sampling_period_us;
22.
23. // Field-Calibrated Tuning Parameters
24. const int NOISE_FLOOR = 4;           // Squelch value to eliminate background static
25. const int MANUAL_GAIN = 4;           // Linear multiplier for high-impedance mic signal
26. const int MAX_EXPECTED = 1000;       // Upper amplitude ceiling for 100% LED fill
27.
28. // Calibrated Decibel Reference Parameters
29. const float DB_FLOOR = 35.0;         // Lower limit for LED initiation (dB)
30. const float DB_CEILING = 75.0;       // Upper limit for full LED ring saturation (dB)
31. const float DB_OFFSET = 40.0;        // Scale alignment factor to approximate standard meters
32.
33. // Graphic State Variables
34. uint16_t globalHue = 0;
35. const int HUE_SPEED = 400;           // Phase shift increment per audio frame
36.
37. // Device Instantiations
38. Adafruit_NeoPixel ring(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
39. ArduinoFFT<double> FFT = ArduinoFFT<double>(vReal, vImag, SAMPLES, SAMPLING_FREQ);
40.
41. void setup() {
42.   Serial.begin(115200);
43.
44.   ring.begin();
45.   ring.setBrightness(40);             // Maintains power rail current limits
46.   ring.show();
47.
48.   analogReadResolution(12);           // Configures ESP32-C6 SAR ADC to 12-bit depth (0 - 4095)
49.   sampling_period_us = round(1000000 * (1.0 / SAMPLING_FREQ)); // Evaluates to 100 microseconds
50. }
51.
52. void loop() {
53.   double sumSquare = 0;
54.   double sampleSum = 0;
55.
56.   // 1. DATA ACQUISITION BLOCK (Strict Time-Domain Interval Sampling)
57.   for (int i = 0; i < SAMPLES; i++) {
58.     unsigned long microseconds = micros();
59.
60.     int raw = analogRead(MIC_PIN);
61.     vReal[i] = (double)raw;
62.     vImag[i] = 0;                     // Imaginary components initialized to zero
63.     sampleSum += raw;
64.
65.     // Enforce uniform sampling frequency interval
66.     while (micros() < (microseconds + sampling_period_us)) { /* Active Wait Lock */ }
67.   }
68.
69.   // 2. SIGNAL CONDITIONING & TIME-DOMAIN ANALYSIS (RMS & dB)
70.   double mean = sampleSum / SAMPLES;
71.   for (int i = 0; i < SAMPLES; i++) {
72.     double centered = vReal[i] - mean; // Dynamic DC offset cancellation
73.     sumSquare += (centered * centered);
74.     vReal[i] = centered;               // Stores pre-filtered data back into FFT array
75.   }
76.
77.   double rms = sqrt(sumSquare / SAMPLES);
78.   double dB = (20 * log10(rms + 0.1)) + DB_OFFSET; // Conversion to log scale
79.
80.   // 3. LEGACY VOLUMETRIC SQUELCH PIPELINE (LED Optimization Mapping)
81.   double signalMax = -9999;
82.   double signalMin = 9999;
83.   for (int i = 0; i < SAMPLES; i++) {
84.     if (vReal[i] > signalMax) signalMax = vReal[i];
85.     if (vReal[i] < signalMin) signalMin = vReal[i];
86.   }
87.   int rawSwing = (int)(signalMax - signalMin);
88.   int cleanSwing = rawSwing - NOISE_FLOOR;
89.   if (cleanSwing < 0) cleanSwing = 0;
90.   int amplifiedSwing = cleanSwing * MANUAL_GAIN;
91.

```

```

92. // 4. FREQUENCY SPECTRUM PROCESSING (FFT Execution)
93. FFT.windowing(FFT_WIN_TYP_HAMMING, FFT_FORWARD);
94. FFT.compute(FFT_FORWARD);
95. FFT.complexToMagnitude();
96.
97. // 5. DIAGNOSTIC SERIAL TELEMTRY STREAM
98. Serial.print("Total_dB:"); Serial.print(dB); Serial.print(",");
99. Serial.print("Vol_Bar:"); Serial.print(amplifiedSwing); Serial.print(",");
100. Serial.print("Bass_Band:"); Serial.print(20 * log10(vReal[2] + 0.1) + DB_OFFSET); Serial.print(",");
101. Serial.print("Mid_Band:"); Serial.print(20 * log10(vReal[8] + 0.1) + DB_OFFSET); Serial.print(",");
102. Serial.print("High_Band:"); Serial.println(20 * log10(vReal[32] + 0.1) + DB_OFFSET);
103.
104. // 6. HARDWARE LEVEL RENDERING
105. int displayLeds = map(amplifiedSwing, 0, MAX_EXPECTED, 0, NUM_LEDS);
106. displayLeds = constrain(displayLeds, 0, NUM_LEDS);
107.
108. if (displayLeds > 0) {
109.     globalHue += HUE_SPEED;
110. }
111.
112. for (int i = 0; i < NUM_LEDS; i++) {
113.     if (i < displayLeds) {
114.         uint16_t ledHue = globalHue + (i * 1000);
115.         ring.setPixelColor(i, ring.gamma32(ring.ColorHSV(ledHue, 255, 255)));
116.     } else {
117.         ring.setPixelColor(i, 0);
118.     }
119. }
120. ring.show();
121. }
122. III
123.

```


Student Tip: After successfully uploading the code using Arduino IDE, open the Serial Monitor (**Ctrl+Shift+M** or **Tools > Serial Monitor**) set to **115200 baud** to view real-time numerical readings.

To see live visual graphs of the DSP calculations, close the Serial Monitor and open the Serial Plotter (**Ctrl+Shift+L** or **Tools > Serial Plotter**). Test your results by playing music on the mic module.



III. Play & Share

- **Build a Graphic Equalizer:** Instead of driving the entire LED ring based on overall volume (amplifiedSwing), use the computed FFT array variables (`vReal[i]`). Divide the ring into distinct physical sectors mapped specifically to **Bass**, **Mids**, and **Highs** to create a true frequency visualizer!
- **Develop a Peak Hold Indicator:** Real audio hardware leaves a single LED lit up at the highest sound level achieved for a brief second before slowly falling back down. Try modifying the hardware loop to track a `peakHoldIndex` that stays floating above the active volume bars.
- **Implement an Automatic Gain Controller (AGC):** If you take the visualizer into a quiet room or a loud concert, the hardcoded limits (`MANUAL_GAIN` and `MAX_EXPECTED`) won't adapt well. Try building a software tracking loop that dynamically rescales your dynamic range boundaries based on the rolling minimum and maximum sound levels captured over the last 10 seconds.